

Data Analysis using Python

06 – Categories and Pivots

Contents

Categorical Data

Binning Data

Pivot Tables

Categorical Data

Categorical data represents values that belong to a finite, discrete set of categories.

Examples include:

- Quality (A++, A, B, Junk).

- Product categories (Electronics, Clothing, Furniture).

- Age groups (Child, Teen, Adult, Senior).

Why Use Categorical Data in Pandas?

Memory Efficiency:

- Reduces memory usage by storing categories as integers.

Improved Performance:

- Faster computations for operations like sorting and filtering.

Logical Ordering:

- Allows ordering of categories (e.g., Small < Medium < Large).

Better Data Integrity:

- Prevents invalid values by restricting data to predefined categories.

Categorical Data

How to Convert a Column to a Category

```
import pandas as pd
```

Sample DataFrame

```
data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],  
        'Gender': ['Female', 'Male', 'Male', 'Female']}
```

```
df = pd.DataFrame(data)
```

Convert Gender to a categorical column

```
df['Gender'] = df['Gender'].astype('category')
```

```
print(df.dtypes)
```

Categorical Data

```
# Ordered Categorical Data
```

```
# Sample DataFrame
```

```
data = {'Size': ['Medium', 'Small', 'Large', 'Medium']}
```

```
df = pd.DataFrame(data)
```

```
# Define a logical order for Size
```

```
size_order = pd.CategoricalDtype(categories=['Small', 'Medium', 'Large'], ordered=True)
```

```
df['Size'] = df['Size'].astype(size_order)
```

```
# Sort the DataFrame by Size
```

```
df = df.sort_values('Size')
```

```
print(df)
```

Categorical Data

```
# Ordered Categorical Data
```

```
# Sample DataFrame
```

```
data = {'Size': ['Medium', 'Small', 'Large', 'Medium']}
```

```
df = pd.DataFrame(data)
```

```
# Define a logical order for Size
```

```
size_order = pd.CategoricalDtype(categories=['Small', 'Medium', 'Large'], ordered=True)
```

```
df['Size'] = df['Size'].astype(size_order)
```

```
# Sort the DataFrame by Size
```

```
df = df.sort_values('Size')
```

```
print(df)
```



	Size
1	Small
0	Medium
3	Medium
2	Large

Categorical Data

Grouping and Aggregating Categorical Data

Sample DataFrame

```
data = {'Product': ['A', 'B', 'A', 'C', 'B', 'C'],  
        'Category': ['Electronics', 'Clothing', 'Electronics', 'Furniture', 'Clothing', 'Furniture'],  
        'Sales': [100, 200, 150, 300, 250, 400]}  
df = pd.DataFrame(data)
```

Convert Category to categorical type

```
df['Category'] = df['Category'].astype('category')
```

Group by Category and calculate total sales

```
grouped_sales = df.groupby(by='Category', observed=False)['Sales'].sum()
```

```
print(grouped_sales)
```



Category	
Clothing	450
Electronics	250
Furniture	700

Binning Data

Binning is the process of grouping numerical data into intervals, or “bins.”

Commonly used to

- Simplify data analysis.

- Identify patterns or distributions.

- Transform continuous variables into categorical ones.

Why Use Binning?

- Data Simplification

 - Reduces the complexity of numerical data by grouping it into intervals.

- Trend Analysis

 - Helps visualize data distributions (e.g., histograms).

- Categorical Transformations

 - Converts numerical data into categories (e.g., age groups).

- Outlier Management:

 - Handles extreme values by grouping them into bins.

Binning Data

```
# Using pd.cut for Equal-Width Bins
```

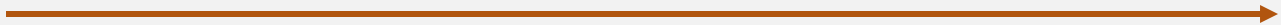
```
import pandas as pd
```

```
# Sample DataFrame
```

```
data = {'Age': [15, 22, 37, 45, 55, 65, 72]}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```



	Age
0	15
1	22
2	37
3	45
4	55
5	65
6	72

```
# Create equal-width bins
```

```
df['Age Group'] = pd.cut(df['Age'], bins=3,  
                        labels=['Young', 'Middle-aged', 'Senior'])
```

```
print(df)
```



	Age	Age Group
0	15	Young
1	22	Young
2	37	Middle-aged
3	45	Middle-aged
4	55	Senior
5	65	Senior
6	72	Senior

Binning Data

```
# Using pd.qcut for Equal-Frequency Bins
```

```
# Create equal-frequency bins
```

```
df['Income Group'] = pd.qcut(df['Age'], q=3, labels=['Low', 'Medium', 'High'])
```

```
print(df)
```



	Age	Age Group	Income Group
0	15	Young	Low
1	22	Young	Low
2	37	Middle-aged	Medium
3	45	Middle-aged	Medium
4	55	Senior	High
5	65	Senior	High
6	72	Senior	High

Binning Data

```
# Binning Sales Data
# Sample DataFrame
sales_data = {'Sales': [100, 200, 300, 400, 500, 600, 700]}
df_sales = pd.DataFrame(sales_data)

# Define bins for sales
bins = [0, 250, 500, 750]
labels = ['Low', 'Medium', 'High']

# Create sales categories
df_sales['Sales Category'] = pd.cut(df_sales['Sales'],
                                    bins=bins,
                                    labels=labels)

print(df_sales)
```

	Sales	Sales Category
0	100	Low
1	200	Low
2	300	Medium
3	400	Medium
4	500	Medium
5	600	High
6	700	High

Pivot Tables

Pivot tables are tools for summarizing and analyzing data by reorganizing and aggregating it.

Similar to pivot tables in Excel, they allow you to:

- Rearrange rows and columns.

- Aggregate data with functions like sum, mean, etc.

- Quickly extract insights from large datasets.

Why Use Pivot Tables in Pandas?

Data Summarization

Quickly generate summaries of large datasets.

Customizable

Flexible grouping and aggregation with user-defined functions.

Multi-Level Analysis

Allows grouping by multiple levels of rows and columns.

Efficient

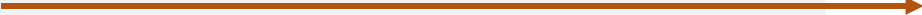
Handles large data efficiently compared to manual aggregation.

Pivot Tables

Basic Syntax

```
pd.pivot_table(data,  
                values=None,  
                index=None,  
                columns=None,  
                aggfunc='mean')
```

```
import pandas as pd  
  
# Sample DataFrame  
data = {  
    'Region': ['East', 'East', 'West', 'West', 'East', 'West'],  
    'Product': ['A', 'B', 'A', 'B', 'A', 'B'],  
    'Sales': [200, 250, 300, 400, 150, 350]  
}  
df = pd.DataFrame(data)  
  
# Create a pivot table  
pivot = pd.pivot_table(df, values='Sales', index='Region',  
                        columns='Product', aggfunc='sum')  
  
print(pivot)
```



Product	A	B
Region		
East	350	250
West	300	750

Pivot Tables

```
# Example 2: Adding Margins (Grand Totals)
# Pivot table with grand totals
pivot_with_totals = pd.pivot_table(df,
                                   values='Sales',
                                   index='Region',
                                   columns='Product',
                                   aggfunc='sum',
                                   margins=True)
```

```
print(pivot_with_totals)
```

Product	A	B	All
Region			
East	350	250	600
West	300	750	1050
All	650	1000	1650

```
# Example 3: Multi-Level Index in Pivot Tables
# Multi-level pivot table
multi_pivot = pd.pivot_table(df,
                              values='Sales',
                              index=['Region', 'Product'],
                              aggfunc='sum')
```

```
print(multi_pivot)
```

Region	Product	Sales
East	A	350
	B	250
West	A	300
	B	750

